



ಮೈಸೂರು ವಿಶ್ವವಿದ್ಯಾನಿಲಯ UNIVERSITY OF MYSORE



MYSORE UNIVERSITY SCHOOL OF ENGINEERING

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Project Phase - II(21AIP81)

Presentation On

“Desktop Voice Assistant Using NLP”

Presented By,
Hariharan S – 21SEAI11
Krupa M C - 21SEAI13
Sameera H P – 21SEAI30
VIII Semester,
Department of AI&ML
MUSE.

Dr. M. S. Govinde Gowda
Director,
MUSE, UOM, Mysuru.

Under Supervision,
Mrs. Manasa K J
Asst. Professor,
Dept. of AI&ML,
MUSE, UOM, Mysuru.

Desktop voice assistant using NLP

- Overview of desktop voice assistant using natural language processing.
- Importance of adapting to evolving NLP functionalities for hand-free experiences of desktop voice assistant.
- Natural language processing techniques that enhance security, reliability and user experience.

Presentation Outline

- Introduction
- Motivation
- Problem Statement
- Related Work
- Proposed System Architecture
- Project Objectives
- Implementation And Challenges
- Solutions
- Evaluation Metrics
- NLP Process
- Real World Applications
- Result and Discussion
- Conclusion
- References

➤ Introduction

A desktop voice assistant uses natural language to understand and respond to your spoken commands, making it easy to interact with your computer hands-free. It helps with everyday tasks like opening apps, searching online, or setting reminders just by talking to it.

➤ Motivation

The motivation behind building a desktop voice assistant is to make computer interaction faster, more intuitive, and accessible, especially for those with physical limitations. It enhances user convenience by allowing hands-free control through natural conversation.

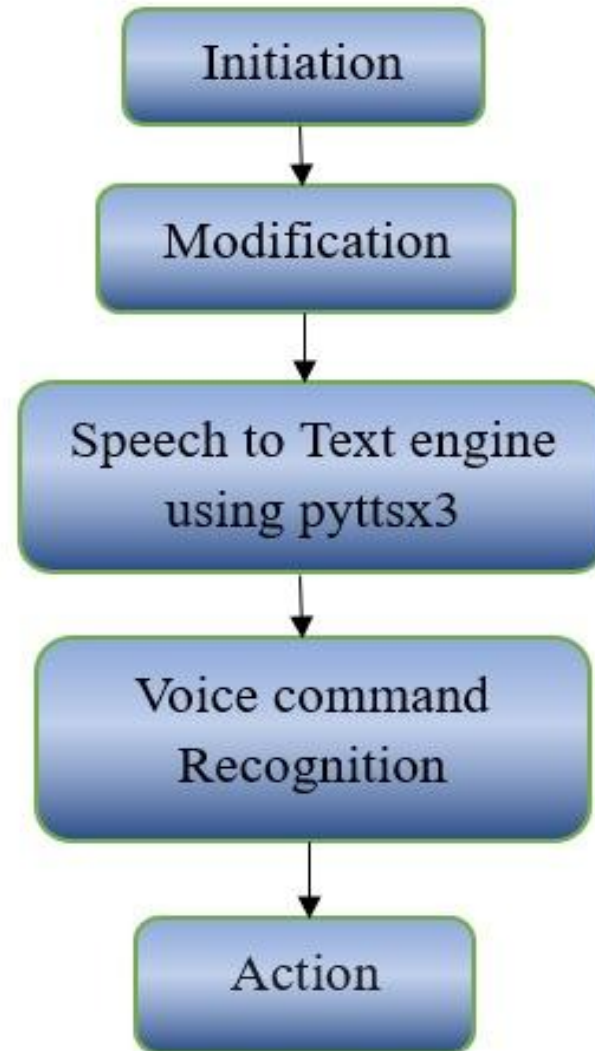
➤ Problem Statement

In today's fast-paced digital world, users interact with computers using traditional input methods like keyboard and mouse, which can be time-consuming, non-intuitive, and inaccessible to differently-abled individuals. There is a growing demand for smart, hands-free systems that allow users to control their devices using natural language. This project addresses the need for an efficient, voice-driven desktop assistant that utilizes Natural Language Processing (NLP) to recognize, interpret, and execute spoken commands, thereby enhancing user convenience, system accessibility, and overall productivity.

➤ Related Work

Existing voice assistants such as Google Assistant, Siri, and Amazon Alexa provide powerful voice-based interaction but are largely cloud-dependent and designed for mobile or smart devices rather than desktop environments. Open-source alternatives like Mycroft and Jasper have introduced offline functionality and modular designs; however, they often require complex setup procedures and lack flexibility for personalized desktop automation tasks. This project builds upon these existing systems by proposing a lightweight, customizable, and desktop focused voice assistant that leverages NLP for intent detection while maintaining offline functionality for enhanced privacy and accessibility.

➤ Proposed System Architecture



1. Initiation

At startup, the assistant initializes core modules (e.g., audio interfaces, configuration files) and optionally warms up the speech engine or wake-word detector to stay alert and responsive.

2. Modification

This preparatory stage configures settings such as audio gain, language preferences, voice selection, or wake-word thresholds to tailor the assistant's behavior and readiness for voice input.

3. Speech-to-Text Engine using pyttsx3

Although pyttsx3 is a text-to-speech (TTS) engine used for speaking responses, the speech-to-text (STT) component often relies on libraries like SpeechRecognition with engines like Google's or offline ones like CMUSphinx. Meanwhile, pyttsx3 converts the assistant's textual answers back into spoken form—all working offline

4. Voice Command Recognition

The transcribed text is then parsed using NLP tools (e.g., regex, spaCy, or NLTK) to detect user intent—such as “open browser” or “play music”—by matching phrases or semantic patterns .

5. Action

Once intent is identified, the assistant triggers the associated function like launching apps, retrieving information, sending emails, or controlling media via system calls or integrated APIs.

6. Response (Text To Speech with pyttsx3)

After executing the command, the result is transformed into natural speech using pyttsx3. You can customize the voice, speaking rate (words per minute), volume, and even save results as audio files all offline.

Objectives

Objective 1:

Demonstrate Real-World Use of NLP and Python Libraries

Showcase practical integration of Python libraries (pyttsx3, speech recognition, pywhatkit, psutil, etc.) to build a working AI system with NLP capabilities.

Objective 2:

Simplify User Interaction with System Resources

Provide an intuitive voice interface to control and access desktop functionalities such as launching apps, checking system info, or browsing the web.

Objective 3:

Capabilities of responding to human feeling To Play songs related to human Mood like, reply for I am sad to hear that if the user says I am sad, if the user is tired or bored play some cheerful musics.

Implementation and Challenges

objective 1:

Accurate Speech Recognition : Background noise and accent variations affect recognition accuracy using libraries like speech_recognition.

NLP Understanding Limitations : Extracting intent from user queries (like “play a song” vs “open a song”) can confuse rule-based or keyword-based NLP.

.

Command Generalization : may phrase commands differently (“open YouTube” vs “launch YouTube”), making hard-coded rules less effective.

Real-Time Response : Managing latency in voice recognition and execution while maintaining a responsive user experience.

Library Compatibility Issues: Integrating multiple Python libraries (e.g., pyttsx3, pywhatkit, BeautifulSoup) can lead to conflicts or crashes.

Solutions implemented for overcoming the challenges:

Noise Handling : Used `adjust_for_ambient_noise()` from `speech_recognition` to filter background noise before listening.

Preprocessing of Commands : Applied basic NLP techniques like tokenization, lowercasing, and stop word removal to standardize input commands.

Intent-Based Matching : Included flexible keyword-based matching with synonyms and partial matching logic to interpret a wide variety of voice inputs.

Efficient Execution Flow : Modular function design ensured smooth transition from speech recognition → command processing → action

Objective 2:

Accessing System-Level Applications : Launching and closing native apps (like Notepad, CMD, Paint) reliably across platforms required careful system calls.

Understanding Natural Language Queries : Users may say “open calculator”, “start calc”, or “launch calculator” making uniform interpretation difficult.

Permission and Security Barriers : Some system-level actions (shutdown, restart) require elevated privileges or may be blocked by OS policies.

Multiple Commands Management : Users may issue commands back-to-back (e.g., “open YouTube, then play music”), making flow management complex.

Dependency on OS-Specific Functions : Implementation differs between Windows, macOS, and Linux, complicating the assistant's portability

Solutions Implemented for Overcoming the Challenges:

Platform-Specific Commands : Used `os.system` and `subprocess.Popen()` with OS checks (via `sys.platform`) to support multi-platform actions.

Voice-to-Command Mapping Logic : Created a mapping dictionary with alternate command phrases for each action to improve recognition consistency.

Safe Execution Handling : For critical operations like shutdown or restart, added confirmation layers and error handling to ensure safety.

Threaded Command Execution : Ensured assistant remains responsive while executing long-running tasks (e.g., opening apps or web browsing).

User Feedback Mechanism : Voice confirmations using pyttsx3 reassure the user that actions (like “Opening Notepad”) have been correctly received and executed

Objective 3:

Understanding Emotional Context from Text : Detecting simple moods like “I am sad” is manageable, but variations like “feeling down today” require deeper sentiment analysis.

Lack of Emotional Tone in Voice Input : Without sentiment analysis or voice emotion recognition, understanding feelings relies solely on literal keywords.

Appropriate Media Response Selection : Matching the right type of music or content (e.g., happy songs for “bored”) with the mood detected.

Avoiding Misinterpretation : Preventing false positives - not every sentence with “sad” or “tired” indicates actual user emotion.

Fallback Response Planning : When a feeling isn't detected, the assistant must still respond meaningfully without seeming unhelpful.

Solutions Implemented for Overcoming the Challenges:

Keyword-Based Mood Detection : Parsed user inputs for phrases like “I am sad”, “I am tired”, “I am bored”, and “I am happy” using if-condition checks.

Emotion-Specific Responses & Actions :

- “I am sad” → Uplifting song playback and reassurance message.
- “I am bored” → Play “funny videos” on YouTube.
- “I am tired” → Relaxing music recommendations.
- “I am happy” → Cheerful acknowledgment.

Integrated YouTube Search via pywhatkit : Triggered direct YouTube content play based on detected mood.

Evaluation Matrix

ROUGE (Recall-Oriented Understudy for Gisting Evaluation):

is a set of metrics used to evaluate the quality of generated text especially in NLP tasks like summarization and response generation.

ROUGE-N: Overlap of n-grams (e.g., unigrams, bigrams) between system and reference.

ROUGE-L: Longest Common Subsequence (LCS) similarity.

ROUGE-W: Weighted LCS that considers sequence length.

ROUGE-S: Skip-bigram co-occurrence

NLP process

Tokenization & Lemmatization

Tokenization :

- Splitting a sentence or paragraph into individual units (tokens) like words or phrases.
- Helps in understanding sentence structure.

Example: Input: “Jarvis is listening carefully”.

Tokens: ["Jarvis", "is", "listening", "carefully", "."]

Lemmatization :

- Converts a word to its base or dictionary form (lemma).
- More meaningful than stemming.

Example: Input: “listening”

Lemma: “listen”

POS Tagging & Named Entity Recognition (NER)

Part-of-Speech (POS) Tagging :

Assigns grammatical categories (noun, verb, adjective, etc.) to each word.

Example : Sentence: “Jarvis plays music”.

Output: [('Jarvis', 'NNP'), ('plays', 'VBZ'), ('music', 'NN')]

Named Entity Recognition (NER) :

Identifies proper names (person, location, date, etc.) in text.

Example: Input: “Elon Musk founded SpaceX in California”.

Output: Elon Musk → PERSON SpaceX → ORGANIZATION California → LOCATION

Stop Word Removal, Chunking & Intent Recognition

Stop Word Removal :

Eliminates common words that do not add significant meaning (e.g., "is", "the").

Before: “Jarvis is a smart assistant”

After: “Jarvis smart assistant”

Chunking :

Combines POS tags into phrases like noun or verb groups.

Example : “The quick brown fox” → Noun Phrase (NP)

Intent Recognition :

Uses NLP to understand the purpose of the user's input.

Example: “Play a happy song” → Intent : PlayMusic
“What’s the weather today?” → Intent : GetWeather

Real-World Applications of NLP

1. Virtual Assistants

- Examples: Siri, Alexa, Google Assistant etc Understand and respond to voice commands using NLP.

2. Speech Recognition

- Converts spoken language into text.
- Used in transcription, voice typing, accessibility tools.

3. Spam Detection

- Classifies emails based on textual patterns.
- NLP helps identify spam, phishing, or promotional messages.

4. Text Summarization

- Automatically summarizes long documents or articles.
- Used in news apps, research tools, legal document analysis.

5. Chatbots

- Automate customer support and answer FAQs.
- NLP enables meaningful and context-aware responses.

6. Search Engines

- Google uses NLP to understand queries and rank pages more accurately

Result and Discussion

The Desktop Voice Assistant's effectiveness was measured using **ROUGE metrics**, evaluating the relevance of its spoken responses to expected outcomes.

key Highlights

- Achieved 80% ROUGE Accuracy, indicating high-quality responses.
- Successfully mapped recognized speech to intended actions.
- Handled noisy environments via `adjust_for_ambient_noise()`.
- Enabled real-time tasks like opening apps, telling jokes, and playing music.

Conclusion

In the era of intelligent systems, voice assistants have emerged as powerful tools that enable seamless human-computer interaction. This project presents a **Desktop Voice Assistant** developed using Python and Natural Language Processing (NLP) techniques.

The assistant is capable of recognizing voice commands, interpreting user intent, and performing a wide range of tasks such as:

- Opening applications
- Providing system information
- Playing music/videos
- Searching and summarizing content
- Responding empathetically to user emotions

- By combining **speech recognition**, **text-to-speech**, and various **Python libraries**, this assistant creates a hands-free, user-friendly experience.
- It serves as a practical implementation of AI-powered automation for everyday desktop use, especially useful for accessibility and productivity.

References

- **A.A.Shaikh** and **V. T. Humbe**, "An Intelligent Voice Assistant Using Deep Learning," 2020 IEEE International Conference on Smart Technologies, Pune, India, 2020, pp. 1–5.
- **V. Patil**, **S. Ingle**, and **A. Raut**, "Smart Personal Assistant Using Speech Recognition," 2019 IEEE International Conference on Intelligent Computing and Control Systems (ICCS), Madurai, India, 2019, pp. 1202–1205.

- **P. N. Kadam** and **P. S. Jadhav**, "Design and Implementation of Voice Controlled AI Assistant," *International Journal of Computer Applications*, vol. 182, no. 22, pp. 24–29, 2018.
- **L. R. Rabiner**, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, Feb. 1989.
- **S. M. Shete** and **A. M. Shete**, "Jarvis: An Intelligent Personal Assistant," 2018 International Conference on Smart City and Emerging Technology (ICSCET), Mumbai, India, 2018, pp. 1–5.

Suggestions/questions



Thank You All

